

R to Go: Translating an IRS 990 Data Pipeline

A practical guide for R programmers encountering Go for the first time

This guide walks through how an IRS 990 XML data pipeline that started life as a handful of R scripts ended up rewritten in Go. It is written for someone who knows R well — tidyverse, purrr, xml2 — and is reading Go code for the first time. Every section pairs an R pattern you already know with its Go equivalent.

Section 1

Why Go?

R is an excellent tool for interactive data exploration, one-off analyses, and prototyping pipelines. But when you want something that runs continuously as a service — polling for new IRS data every week, writing to a database, resuming where it left off after a restart — R's interactive model starts to feel like the wrong fit.

R is great for...	Go is better for...
Interactive exploration in the console	Long-running background services
Quick scripts you run once	Programs that run continuously for months
Results that live in memory	Results persisted to a database
Dynamic, flexible data shapes	Strictly-typed, predictable data contracts

Go is **statically typed** (every variable has a declared type, checked at compile time), **compiled** (a single binary you ship), and has **built-in concurrency primitives** — goroutines and channels are part of the language, not a library. The ideas from your R pipeline translate directly; the syntax just looks different.

Section 2

Project Structure

In R, a pipeline is typically a flat set of .R script files you run in sequence. In Go, code is organized into **packages** — each directory is a package. Here is how this project maps to the R scripts:

R	Go
<pre># R scripts (flat): schedule_J_parser.R schedule_J_cleaner.R xml_parse_code.R</pre>	<pre>// Go packages (directories): cmd/main.go // entry point internal/fetcher/ // HTTP downloads internal/parser/ // XML parsing internal/models/ // shared data types internal/db/ // database writes</pre>

A **package** in Go is like a namespace. Code in `internal/parser/` belongs to package `parser`. Other packages import it by path: `"github.com/.../internal/parser"`. Functions that start with an uppercase letter are exported (public); lowercase names are package-private — there is no `export()` step.

Section 3

Static Typing — The Biggest Mental Shift

In R, a function can return anything — a character vector, a matrix, a list — and you figure out the shape at runtime. In Go, every function's return type is declared upfront, and the compiler rejects code that violates it. The payoff is that you can never accidentally pass the wrong thing to a function.

R	Go
<pre>xml_comp_parser <- function(xml) { ein <- xml_file %>% xml_find_first(xpath="//d1:EIN") %>% xml_text() values <- xml_file %>% xml_find_all("//d1:IRS990ScheduleJ") %>% xml_children() %>% xml_text() # shape is whatever comes back mat <- matrix(c(ein, taxyr, values), ...) return(mat) # could be anything }</pre>	<pre>// Shape declared upfront as a struct type personComp struct { PersonNm string `xml:"PersonNm"` TitleTxt string `xml:"TitleTxt"` TotalCompFilingOrgAmt string `     `xml:"TotalCompFilingOrgAmt"` } // Parser returns a known type or an error func Parse(data []byte, ...) (*Result, error) { var doc return990 xml.Unmarshal(data, &doc) return &Result{...}, nil }</pre>

The `CompensationRecord` struct in `internal/models/models.go` is the Go equivalent of the column selection you do in R with `str_detect(names(.), "PersonNm.*|TitleTxt.*|Total.*")` — except the shape is fixed at compile time, not

discovered at runtime.

Section 4

Error Handling

R's `purrr::possibly()` wraps a function so that any error silently becomes NA. This is convenient for exploration, but in a long-running service you want to know *which* files failed and *why*. Go makes errors explicit: every function that can fail returns two values — the result and an error.

R	Go
<pre># Silently swallow errors, return NA comp <- files %>% map(possibly(xml_comp_parser, otherwise = NA_real_)) # You won't know which files failed # unless you check for NA afterward</pre>	<pre>// Every failure is explicit result, err := parser.Parse(xmlData, objectID, companyName, city, state, ntee,) if err != nil { slog.Warn("parse failed", "object_id", objectID, "err", err) return nil // skip, log, move on }</pre>

The `if err != nil` pattern is repetitive, and Go programmers joke about it. But its advantage is visibility: every failure path is explicitly handled at the call site. There are no silent NAs propagating through your pipeline undetected.

Section 5

XML Parsing — XPath vs. Struct Tags

In R, you write XPath strings at runtime to extract values from XML nodes. In Go, you declare the mapping *once* as struct field tags, and the standard library figures out the rest when you call `xml.Unmarshal()`. The struct tags are checked at compile time; there are no typos in XPath strings that you only discover when you run the code.

R	Go
<pre># XPath strings evaluated at runtime ein <- xml_file %>% xml_find_first(xpath = '//d1:EIN') %>% xml_text() name <- xml_file %>% xml_find_all(xpath = '//d1:PersonNm') %>% xml_text()</pre>	<pre>// Struct tags declared once; no XPath type personComp struct { PersonNm string `xml:"PersonNm"` TitleTxt string `xml:"TitleTxt"` BaseComp string \ `xml:"BaseCompensationFilingOrgAmt"` } // One call maps the whole document var doc return990 xml.Unmarshal(data, &doc) // doc.ReturnData.ScheduleJ.Persons // is now a []personComp – fully parsed</pre>

The backtick notation — `xml:"PersonNm"` — is a **struct tag**. It is a string literal attached to a field that the encoding/xml package reads via reflection at runtime. Think of it as column metadata that lives right next to the field declaration, rather than in a separate XPath expression somewhere else in the code.

Section 6

The Pipeline Pattern — map() vs. Goroutines

The R pipeline uses `purrr::map()` to apply `xml_comp_parser` to every file sequentially. Go replaces this with a **worker pool**: a fixed number of goroutines running in parallel, bounded by a semaphore so you do not overwhelm the IRS server or your database.

R	Go
<pre># Sequential – one file at a time comp <- files %>% map(possibly(xml_comp_parser, otherwise = NA_real_)) # map() runs in the R process, # one iteration at a time.</pre>	<pre>// Concurrent – workers run in parallel sem := make(chan struct{}, workers) g, ctx := errgroup.WithContext(ctx) for _, entry := range entries { entry := entry // capture loop var sem <- struct{}{} g.Go(func() error { defer func() { <-sem }() return processXML(ctx, entry) }) } return g.Wait()</pre>

A **goroutine** is a lightweight concurrent function — not a thread, but something Go schedules across your CPU cores. `go func() { ... }()` starts one. The **semaphore channel** (`sem`) limits how many goroutines run at once: sending to a full channel blocks until a slot frees up; the deferred receive releases the slot when the goroutine finishes. `errgroup` collects errors from all goroutines and cancels the context if any fail.

Section 7

New Concepts: HTTP and the Database

The original R scripts assumed files were already downloaded manually to a local directory, and results lived in memory until the R session ended. The Go pipeline handles both fetching and persistence natively.

Fetching data with net/http

Go

```
// fetcher.go – simplified
func (f *Fetcher) get(
    ctx context.Context, url string,
) (io.ReadCloser, error) {
    req, _ := http.NewRequestWithContext(
        ctx, http.MethodGet, url, nil)
    resp, err := f.client.Do(req)
    if err != nil { return nil, err }
    return resp.Body, nil
}

// IRS ZIPs are 2-3 GB; client timeout: 4h
```

Writing results to Postgres with pgx

Go

```
// db/writer.go — batch insert via COPY
func InsertCompensation(
    ctx context.Context,
    pool *pgxpool.Pool,
    records []*models.CompensationRecord,
) error {
    _, err := pool.CopyFrom(ctx,
        pgx.Identifier{"schedule_j_compensation"},
        cols,
        pgx.CopyFromRows(rows),
    )
    return err
}
```

Resume where you left off. Before downloading any ZIP archive, the pipeline calls `db.GetProcessedIDs()` to bulk-check which filings are already in the database. ZIPs can be 2-3 GB — skipping ones you have already processed saves hours. The R script had no equivalent: if it crashed, you started over.

Section 8

Key Go Concepts Glossary

Quick reference for terms that appear throughout the codebase:

package	A directory of .go files that share a namespace. The first line of every file declares it: <code>package parser</code> . Closest R analogy: a package you load with <code>library()</code> .
func	A function declaration. <code>func Parse(data []byte) (*Result, error)</code> — name, parameters with types, return types. All explicit, no guessing.
struct	A named collection of fields with declared types. Like a row definition or a named list with a fixed schema. Used everywhere data has a predictable shape.
interface	A set of method signatures. Any type that implements those methods satisfies the interface — no explicit implements keyword needed. Think of it as duck typing, but checked at compile time.
goroutine	A lightweight concurrent function started with the <code>go</code> keyword. The Go runtime schedules thousands of them across OS threads. Much cheaper than a thread; roughly analogous to a future or promise.
channel	A typed pipe for sending values between goroutines. <code>ch <- v</code> sends; <code>v := <-ch</code> receives. Used as a semaphore here to cap concurrency.

error	A built-in interface with one method: <code>Error()</code> string. Returned as the last value from functions that can fail. Check with <code>if err != nil</code> .
defer	Schedules a function call to run when the surrounding function returns, regardless of how it returns (normal, panic, etc.). Used to close files and release semaphore slots.
context	Carries deadlines, cancellation signals, and request-scoped values across API boundaries. Pass it as the first argument to any function that does I/O. Cancel it to abort in-flight work.

The full source is at github.com/BatonVatrushka/irs_990_xml_parse. The R scripts that started this pipeline are still in the repository root — `schedule_J_parser.R`, `schedule_J_cleaner.R` — if you want to compare the original R approach with the Go implementation side by side.